

Projekta darbs vetcare-lite

Izglītības iestāde: Rīgas Tālmācības Vidusskola

Kurss: Programmēšana II / Projekta darbs

Darba nosaukums: vetcare-lite

Darba autors: Elīza Sofija Čifarati 12.kl

Saturs

1. Ievads
2. Projekta mērķis, attiecīgie uzdevumi
3. Izmantotās tehnoloģijas
4. Sistēmas apraksts, funkcionalitāte
5. Datu bāzes struktūra
6. Lietotāju autentifikācija un drošība
7. Administratora panelis
8. API integrācija
9. Projekta izstrādes gaita
10. Testēšana
11. Secinājumi
12. Izmantotie avoti

1. Ievads

Mūsdienās veterinārajām klīnikām nepieciešamas ērtas un efektīvas sistēmas klientu, dzīvnieku un pierakstu pārvaldīšanai. Daudzas nelielas klīnikas joprojām izmanto papīra pierakstus vai vienkāršas tabulas, kas nav drošas un ir grūti pārvaldāmas.

Šī projekta ietvaros tika izstrādāta tīmekļa sistēma “vetcare-lite”, kas paredzēta nelielas veterinārās klīnikas darbības organizēšanai. Sistēma ļauj reģistrēt darbiniekus, pārvaldīt klientus un viņu dzīvniekus, kā arī veidot vizīšu pierakstus.

Projekts tika izstrādāts Python programmēšanas valodā, izmantojot Flask tīmekļa ietvaru un SQLite datu bāzi.

2. Projekta mērķis, attiecīgi uzdevumi

Projekta mērķis bija izveidot vienkāršu veterinārās klīnikas vadības sistēmu ar lietotāju autentifikāciju, klientu un mājdzīvnieku pārvaldību, kā arī vizīšu rezervēšanas iespējām.

Projektā vēlējos vairāk fokusēties uz backend izstrādi, datu bāzu izmantošanu un lietotāju piekļuves kontroli.

Lai sasniegtu projekta mērķi, tika izvirzīti šādi uzdevumi:

- izveidot Flask tīmekļa lietotni;
- izveidot SQLite datu bāzi;
- realizēt lietotāju sistēmu ar paroles šifrēšanu;
- izveidot administratora paneli;
- realizēt klientu un dzīvnieku pārvaldību;
- pievienot vizīšu rezervēšanas funkciju;
- integrēt laikapstākļu API;
- nodrošināt piekļuves kontroli un lietotāju apstiprināšanu;
- notestēt sistēmas darbību.
-

3. Izmantotās tehnoloģijas

Python - galvenā programmēšanas valoda, jo tā ir piemērota backend un nelielu tīmekļa sistēmu izstrādei.

Flask - izmantots sistēmas maršrutiem (routes), loģikai un lietotāju pieprasījumu apstrādei.

SQLite - izmantota kā datu bāze projekta datu glabāšanai.

SQLAlchemy - izmantots ērtākai datu bāzes modeļu un relāciju izveidei.

Werkzeug - izmantots paroļu hashēšanai un pārbaudei login sistēmā.

Requests - izmantota API pieprasījumu veikšanai.

Open-Meteo API - izmantots aktuālo laikapstākļu datu iegūšanai

4. Sistēmas apraksts, funkcionalitāte

Sistēma sastāv no vairākām galvenajām daļām:

- lietotāju autentifikācijas sistēmas;
- administratora paneļa;
- klientu un mājdzīvnieku pārvaldības;
- vizīšu rezervēšanas sistēmas;
- laikapstākļu API integrācijas.

Jauni lietotāji var reģistrēties sistēmā, taču piekļuve tiek piešķirta tikai pēc administratora apstiprinājuma.

Pēc pieslēgšanās lietotājs var pievienot un rediģēt klientus, pārvaldīt mājdzīvniekus, kā arī rezervēt veterinārās vizītes. Admin papildus pieejama lietotāju pārvaldība un kontu apstiprināšana.

Funkcionalitāte :

Lietotāju sistēma

- Reģistrācija
- Pieslēgšanās
- Izrakstīšanās
- Administratora apstiprinājums

Klientu pārvaldība

- Klientu pievienošana

- Rediģēšana
- Dzēšana
- Meklēšana pēc vārda vai telefona

Dzīvnieku pārvaldība

- Dzīvnieku pievienošana
- Rediģēšana
- Dzēšana

Vizīšu sistēma

- Vizīšu rezervēšana
- Datuma validācija
- Vizīšu saraksts

Admin funkcijas

- Lietotāju pārvaldība
- Kontu apstiprināšana
- Kontu rediģēšana
- Kontu dzēšana

5. Datu bāzes struktūra

Projektā tika izveidotas četras galvenās tabulas: User, PetOwner, Pet, Appointment.

User tabula

Glabā sistēmas lietotāju informāciju:

- id;
- username;
- password_hash;
- is_approved.

PetOwner tabula

Glabā klientu informāciju:

- id;
- name;
- phone;
- email.

Pet tabula

Glabā mājdzīvnieku informāciju:

- id;
- name;
- species;
- breed;
- owner_id.

Appointment tabula

Glabā veterināro vizīšu informāciju:

- id;
- datetime;
- reason;
- pet_id.

6. Lietotāju autentifikācija un drošība

Projektā tika ieviesta sesiju autentifikācija.

Pēc veiksmīgas pieslēgšanās sistēma saglabā lietotāja id sesijā.

Drošības nolūkos paroles netiek glabātas atklātā tekstā. Tās tiek šifrētas, izmantojot Werkzeug bibliotēku.

Sistēmā tika realizēta arī “Zero-Trust” pieeja:

- jauni lietotāji tiek izveidoti kā “pending”;
- līdz administratora apstiprinājumam nav piekļuve nekam.

Papildus tika ieviesti:

- admin piekļuves ierobežojumi;

- sesiju pārbaudes;
- apstiprināšanas sistēma;
- paroles redzamības funkcija;
- dzēšanas apstiprināšanas logi.

7. Admin panelis

Admin ir pieejams īpašs panelis darbinieku pārvaldībai.

Panelī iespējams:

- apskatīt visus lietotājus;
- redzēt kontu statusu;
- apstiprināt jaunus darbiniekus;
- rediģēt lietotājus;
- dzēst lietotāju kontus.

Drošības nolūkos galveno administratora kontu nav iespējams izdzēst.

8. API integrācija

Projektā tika izmantots Open-Meteo API.

API tiek izmantots, lai iegūtu aktuālo temperatūru Rīgā.

Balstoties uz temperatūru, sistēma ģenerē veterināros ieteikumus (pie siltāka laika tiek parādīts brīdinājums par ērcēm; ziemā tiek brīdināts par sāls ietekmi uz dzīvnieku ķepām.)

Ja API nav pieejams, sistēma parāda rezerves ziņojumu

9. Projekta izstrādes gaita

Sākmā tika izveidota Flask projekta struktūra.

Tālāk tika izstrādāti datu bāzes modeļi un savienojums ar SQLite.

Pēc tam tika realizēta:

1. klientu pārvaldība;

2. dzīvnieku sistēma;
3. vizīšu funkcionalitāte.
4. lietotāju sistēma;
5. administratora sistēma;

Vēlāk projektam tika pievienota:

1. meklēšanas funkcija;
2. API integrācija;
3. administratora panelis.
4. paroles redzamības funkcija;
5. apstiprinājuma logi dzēšanai;

Izstrādes laikā tika novērstas vairākas kļūdas, piemēram:

- SQLAlchemy relāciju kļūdas;
- sintakses kļūdas;
- datuma validācijas problēmas;
- vairākas administratora paneļa darbības kļūdas (paneļa neparādīšanās dashboard, problēmas ar lietotāju paroles maiņu u.c.)

10. Testēšana

Sistēma tika testēta lokāli, izmantojot Flask izstrādes serveri.

Testēšana tika veikta pakāpeniski projekta izstrādes laikā. Pēc katras jaunas funkcijas pievienošanas tika pārbaudīts, vai tā darbojas korekti un neizraisa kļūdas citās sistēmas daļās.

Tika testētas šādas funkcijas:

- lietotāju reģistrācija;
- pieslēgšanās un izrakstīšanās;
- administratora piekļuves kontrole;
- lietotāju apstiprināšana;
- klientu pievienošana, rediģēšana un dzēšana;
- mājdzīvnieku pievienošana, rediģēšana un dzēšana;
- vizīšu rezervēšana;
- meklēšanas funkcionalitāte;
- API datu iegūšana;
- paroles redzamības funkcija;
- dzēšanas apstiprinājuma logi.

Projekta beigās tika veikta arī pilna sistēmas funkcionalitātes pārbaude, kuras laikā tika pārbaudīta dažādu lietotāju darbība sistēmā.

Tika pārbaudīts:

- administratora un parasto lietotāju login;
- lietotāju apstiprināšana;
- datu pievienošana un dzēšana;
- datu rediģēšana;
- piekļuves ierobežojumi;
- sesiju darbība starp dažādiem lietotājiem.

Testēšanas laikā tika atrastas un novērstas vairākas kļūdas administratora panelī un datu bāzes relācijās.

11. Secinājumi

Izstrādājot projektu “vetcare-lite”, ieguvu daudz praktiskāku izpratni par Flask izmantošanu un backend sistēmu izstrādi kopumā.

Pirms projekta jau zināju dažus pamatus, taču darba gaitā kļuva daudz pārliecinātāka par datu bāzu izmantošanu, sesijām, autentifikāciju un API integrēšanu projektā. Paralēli apguvu arī vairākus jaunus rīkus un konceptus, piemēram, Werkzeug paroļu hashēšanu un laikapstākļu API izmantošanu.

Izstrādes laikā nācās risināt dažādas kļūdas un loģikas problēmas, īpaši administratora paneļa un datu bāzes relāciju daļā, tāpēc projekts palīdzēja attīstīt arī problēmu risināšanas prasmes.

Nākotnē sistēmu būtu iespējams papildināt ar modernāku dizainu, e-pasta paziņojumiem, kalendāra integrāciju un klientu portālu.

12. Izmantotie avoti

1. Python – [w3school python, specifically loops, string formatting and file handling moduli](#)
2. Flask dokumentācija – [lielāko tiesu skolas materiāli](#), <https://www.geeksforgeeks.org/python/flask-tutorial/> (daudz noderīgi linki rakstiem projekta realizēšanai kopumā)
3. Open-Meteo API – <https://open-meteo.com/>, <https://www.youtube.com/watch?tut+weather>
4. Werkzeug – <https://werkzeug.palletsprojects.com/>, <https://youtube.com/playlist+werk>

Youtube video materiāli papildus Flask, API un backend izstrādes konceptu izpratnei. : Traversy Media, Programming with Mosh, and Web Dev Simplified